ChatGPT

Ajuda relatório Java POO

Você

ola chat tudo joia? preciso de ajuda com relatorio de java poo, nao me de a resposta l ogo de cara, me ajuda a pensar. Relatorio de paradigmas da programação

ChatGPT

Opa, Caio! Tudo joia 😄

Manda o primeiro exercício ou a dúvida de Java POO. Vou te ajudar a **interpretar o enunciado e construir o raciocínio**, com perguntas e dicas, sem entregar a solução pronta.

Você

```
class Pessoa {
    private String nome;

    public Pessoa(String nome) {
        nome = nome;
    }

    public String getNome() {
        return nome;
    }
} oq ta errado??

public class Main {
    public static void main(String[] args) {
        Pessoa pessoa = new Pessoa("caio");
        System.out.println(pessoa.getNome());
    }
}
```

Leve sua curiosidade mais longe

Escaneie para baixar o aplicativo do ChatGPT

ChatGPT

O problema está nesta linha do construtor:

```
nome = nome;
```

Você tem **duas coisas chamadas `nome`**:

- O atributo da classe: `private String nome;`
- O parâmetro do construtor: `Pessoa(String nome)`

Dentro do construtor, quando você escreve apenas `nome`, Java entende que está falando do **parâmetro**. Então essa linha atribui o parâmetro a ele mesmo, e o atributo continua com o valor inicial `null`.

Lembra do `this`? Ele representa o objeto atual. **Como você usaria `this` para indicar que o lado esquerdo da atribuição é o atributo do objeto?**

Você

uma classe filha pode acessar diretamente os atributos private da classe pai? por que esse codigo da erro?

```
class Pessoa {
    private String nome = "caio";
}

class Aluno extends Pessoa {
    public void apresentar() {
        System.out.println(nome);
    }
}
```

ChatGPT

Não. **Um atributo `private` só pode ser acessado diretamente dentro da própria classe que o declarou.**

Mesmo com `Aluno extends Pessoa`, o método `apresentar()` está dentro de `Aluno`, e não de `Pessoa`. Por isso, tentar acessar `nome` diretamente causa erro.

Isso faz parte do **encapsulamento**: a classe `Pessoa` controla o acesso aos seus dados.

Leve sua curiosidade mais longe
Escaneie para baixar o aplicativo do ChatGPT

Pensa no `getNome()` do exemplo anterior: **se `Pessoa` tivesse esse método como `public`, a classe `Aluno` conseguiria chamá-lo dentro de `apresentar()`?**

Você

```
class Pessoa {
    private String nome;

    public Pessoa(String nome) {
        this.nome = nome;
    }
}

class Aluno extends Pessoa {
}
```

por que a classe Aluno apresenta erro? uma classe filha precisa ter construtor?

ChatGPT

A classe filha não precisa sempre declarar um construtor, mas o construtor dela precisa chamar um construtor válido da classe pai.

Como você não escreveu um construtor em `Aluno`, o Java cria um automaticamente, equivalente a:

```
Aluno() {
    super();
}
```

O `super()` tenta chamar um construtor de `Pessoa` **sem parâmetros**. Porém, você declarou apenas:

```
public Pessoa(String nome)
```

Por isso ocorre o erro: não existe `Pessoa()` para ser chamado. E **construtores não são herdados**.

Pensa assim: para criar um aluno, primeiro precisamos inicializar a parte dele que é uma pessoa. Nesse código, isso exige um nome.

Leve sua curiosidade mais longe
Escaneie para baixar o aplicativo do ChatGPT

Como você faria um construtor em `Aluno` receber esse nome e passá-lo ao construtor de `Pessoa` usando `super(...)` ?

Você

por que nao consigo criar esse objeto? para que serve uma classe abstrata?

```
abstract class Animal {
    public abstract void emitirSom();
}

public class Main {
    public static void main(String[] args) {
        Animal animal = new Animal();
        animal.emitirSom();
    }
}
```

ChatGPT

O erro está em `new Animal()` : **uma classe abstrata não pode ser instanciada diretamente**.

Ela serve como uma **base para outras classes**, reunindo características comuns e definindo métodos que as classes filhas concretas precisam implementar. Ela também pode ter atributos, construtores e métodos com código.

Neste trecho:

```
public abstract void emitirSom();
```

Você declara que todo animal deve emitir um som, mas **não define como esse som será emitido**. Repara que o método termina com `;` , sem um corpo `{ }` .

Pensa em uma classe `Cachorro` que herda de `Animal` : **qual método ela precisaria implementar para permitir a criação de um objeto com `new Cachorro()` ?**

Você

por que esse metodo nao esta sobrescrevendo o metodo da classe pai? qual a diferenca entre sobrescrita e sobrecarga?

Leve sua curiosidade mais longe
Escaneie para baixar o aplicativo do ChatGPT

```
class Animal {
    public void emitirSom() {
        System.out.println("som");
    }
}

class Cachorro extends Animal {
    @Override
    public void emitirSom(String som) {
        System.out.println(som);
    }
}
```

ChatGPT

O problema é que você **adicionou um parâmetro** ao método da classe filha:

```
emitirSom()           // classe pai
emitirSom(String som) // classe filha
```

Para ocorrer **sobrescrita**, o método da filha precisa manter o mesmo nome e os mesmos tipos de parâmetros do método da classe pai. Assim, a filha fornece um novo comportamento para aquele método.

Já a **sobrecarga** acontece quando métodos têm o mesmo nome, mas parâmetros diferentes em quantidade ou tipo.

No seu código, você criou uma **sobrecarga**: `Cachorro` tem o método herdado sem parâmetros e outro que recebe uma `String`.

O `@Override` pede ao compilador para verificar se existe uma sobrescrita. Como não existe, ele aponta o erro.

Se você quer que o cachorro emita `"au au"` ao chamar `emitirSom()` sem argumentos, o que precisaria mudar na declaração do método da filha?

Leve sua curiosidade mais longe
Escaneie para baixar o aplicativo do ChatGPT

Você

por que esse codigo com interface da erro, mesmo tendo implementado o metodo?

```
interface Movel {
    void mover();
}

class Carro implements Movel {
    void mover() {
        System.out.println("andando");
    }
}
```

ChatGPT

O método foi implementado, mas **a visibilidade dele está diferente da exigida pela interface**.

Na interface, esta declaração:

`void mover();`

é implicitamente `public abstract`. Ou seja, o método precisa ser acessível publicamente.

Já em `Carro`:

`void mover() {`

a ausência de um modificador deixa o método com acesso de **pacote**: ele só pode ser acessado por classes do mesmo pacote.

Ao implementar uma interface, você **não pode reduzir a visibilidade do método**.

Pensando nisso: **qual modificador você precisa colocar antes de `void mover()` em `Carro` para cumprir o contrato da interface?**

Você

quando devo usar uma interface e quando devo usar uma classe abstrata???

Leve sua curiosidade mais longe
Escaneie para baixar o aplicativo do ChatGPT

ChatGPT

Use **interface** quando quiser definir um **contrato de comportamento**: o que uma classe deve conseguir fazer.

Por exemplo, `Carro`, `Pessoa` e `Robo` podem implementar `Movel`. São coisas diferentes, mas todas sabem executar `mover()`.

Use **classe abstrata** quando tiver classes relacionadas que precisam compartilhar **atributos, inicialização ou código comum**.

Por exemplo, `Cachorro` e `Gato` podem herdar de `Animal`, que guarda `nome`, implementa `dormir()` e deixa `emitirSom()` para cada filho implementar.

Interface	Classe abstrata
Define um contrato de comportamento	Define uma base comum para classes relacionadas
Uma classe pode implementar várias	Uma classe só pode estender uma classe
Não possui construtor nem atributos de instância	Pode possuir construtor e atributos de instância

Interface também pode ter métodos com implementação, como os métodos `default`. Então a diferença não é simplesmente "uma tem código e a outra não".

Para pensar: se `Carro` e `Peixe` precisam ter um método `mover()`, mas cada um faz isso de um jeito e não precisam compartilhar dados, **qual opção parece mais adequada?**

Leve sua curiosidade mais longe
Escaneie para baixar o aplicativo do ChatGPT